

**COMMUNITY AND SPECIAL SCHOOLS:
A JOURNEY OF INCLUSION**

A COMMUNITY CONNECT PROJECT REPORT

Submitted by

**SIDHARTH P NAIR (RA2311026050087)
SARANG A R (RA2311026050113)**

In Partial Fulfillment of the Requirements
for the Degree of

BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING WITH SPECIALIZATION IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**



**SCHOOL OF COMPUTING
FACULTY OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI – 621105**

July 2025



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
TIRUCHIRAPPALLI
COLLEGE OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified as the bonafide record of work done by SIDHARTH P NAIR (RA2311026050087) and SARANG A R (RA2311026050113) of 5TH SEMESTER/3RD YEAR, B.Tech. COMPUTER SCIENCE AND ENGINEERING WITH SPECIALISATION IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING Programme in the project-based course **21GNP301L, Community Connect** at **SRM Institute of Science and Technology, Tiruchirappalli** during the Academic Year 2024-2025.

Faculty In-charge

Head of the Department

Submitted for the End Semester Examination held on 29TH JULY 2025

Examiner 1

Examiner 2

CERTIFICATE FROM NGO



HUME CENTRE FOR ECOLOGY AND WILDLIFE BIOLOGY

4th Cross Rd, Subhash Nagar, Kalpetta, Wayanad Kerala 673121, Office : 085929 59380, www.humecentre.in

SCIENCE ACROSS BORDERS

HC-ADM-64/2025

19.07.2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Sidharth P Nair has successfully completed an internship of 110 hours at Hume Centre for Ecology and Wildlife Biology bearing DARPAN ID KL/2017/0170486 as part of the project titled "A GIS Based Rainfall Mapping System."

Regards



C K Vishnudas
Executive Director
Hume Centre

A non-profit research institute working on Ecology, Climate Change, Food Systems and Science Education



HUME CENTRE FOR ECOLOGY AND WILDLIFE BIOLOGY

4th Cross Rd, Subhash Nagar, Kalpetta, Wayanad Kerala 673121, Office : 085929 59380, www.humecentre.in

SCIENCE ACROSS BORDERS

HC-ADM-65/2025

19.07.2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Sarang A R has successfully completed an internship of 110 hours at Hume Centre for Ecology and Wildlife Biology bearing DARPAN ID KL/2017/0170486 as part of the project titled "A GIS Based Rainfall Mapping System."

Regards



C K Vishnudas
Executive Director
Hume Centre

A non-profit research institute working on Ecology, Climate Change, Food Systems and Science Education

PROBLEM STATEMENT

Rainfall plays a crucial role in agriculture, water resource management, and disaster preparedness. However, rainfall patterns exhibit **significant spatial and temporal variability**, which directly impacts crop planning, irrigation management, and flood risk assessment in a region. In many areas, rainfall data is collected through rain gauge stations and weather monitoring centers, but **this data is often available only in tabular formats or as isolated numeric values**, making it difficult to visualize and interpret rainfall distribution effectively across different geographic regions.

Without **spatial visualization of rainfall data**, it becomes challenging for farmers to plan cropping patterns based on water availability, for government agencies to identify drought or flood-prone areas, and for water resource managers to plan the efficient use of water resources. Additionally, the lack of spatial rainfall analysis hinders researchers and policymakers from understanding regional climate variability and taking timely action during extreme rainfall events.

The traditional methods of analyzing rainfall data do not support **layering with other geographic information such as land use, soil type, or elevation data**, which is essential for comprehensive planning and disaster risk reduction.

To address these challenges, there is a need to develop a **GIS-Based Rainfall Mapping System** that can:

- Collect and integrate rainfall data with corresponding geographic coordinates for each observation point.
- Store and manage rainfall datasets efficiently for temporal and spatial analysis.
- Visualize rainfall distribution using GIS-based mapping techniques such as choropleth maps, contour maps, and time-series animations.
- Analyze spatial and temporal patterns in rainfall to identify high and low rainfall zones, seasonal variations, and potential flood or drought risk areas.
- Provide actionable insights to stakeholders such as farmers, water resource planners, disaster management authorities, and researchers for effective planning and informed decision-making.

Using **Geographic Information Systems (GIS)** for rainfall mapping will enhance the understanding of rainfall patterns across different regions, allowing for **data-driven decisions in agriculture, water management, and climate adaptation planning**. The system will enable effective visualization, interpretation, and analysis of rainfall data, thereby contributing to sustainable resource management and disaster preparedness efforts.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	PROBLEM STATEMENT	v
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1	INTRODUCTION	10
	1.1 Introduction	10
	1.2 Objective	10
	1.3 Scope	10
	1.4 Motivation	11
2	TECHNICAL APPROACH	12
3	SYSTEM ARCHITECTURE	13
4	TECHNICAL MODULES IMPLEMENTED	14
5	IMPLEMENTATION	16
	5.1 Rainfall Data Collection via Whatsapp	16
	5.2 Climate Log	18
	5.3 Python Script for Structuring Whatsapp Rainfall Data	19
	5.4 Python GIS Rainfall Mapping	21
	5.5 Grid Mapping	22
	5.6 Rainfall Grip Viewer	23
	5.7 Requirements	26
	5.8 run_mapping_pipeline.bat	26
6	RESULT AND DISCUSSION	27

7	CONCLUSION	29
	COMMUNITY WORK RELEVANCE TO NGO	30
	EXPECTED DELEVERABLES	31
	PROPOSED BUDGET	34
	CONCLUSION AND FUTURE WORK	35
	REFERENCES	36

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	Rainfall Data Collection via Whatsapp	15
5.2	Climate Log	15
5.3	Python Script for Structuring Whatsapp Rainfall Data	17
5.4	Python GIS Rainfall Mapping	19
5.5	Grid Mapping	19
5.6	Rainfall Grid Viewer	22
5.7	Requirements	23
5.8	run_mapping_pipeline.bat	23
6.1	Rainfall Map	25

LIST OF ABBREVIATIONS

API	-	Application Programming Interface
CSV	-	Comma Separated Values
GIS	-	Geographic Information System
GPS	-	Global Positioning System
IoT	-	Internet of Things
KSDMA	-	Kerala State Disaster Management Authority
KSREC	-	Kerala State Remote Sensing and Environment Centre
NGO	-	Non-Governmental Organization
PNG	-	Portable Network Graphics
SDG	-	Sustainable Development Goals
SMS	-	Short Message Service
UN	-	United Nations

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Rainfall is one of the most crucial parameters influencing agriculture, water resource management, disaster preparedness, and climate studies, especially in a monsoon-dependent country like India. The distribution of rainfall is highly variable both spatially and temporally, leading to challenges in planning agricultural activities, managing water resources, and preparing for floods or droughts. Traditionally, rainfall data is collected using rain gauges and meteorological instruments, and the results are presented in tabular or numerical formats, making it difficult for planners, farmers, and authorities to interpret and use this data effectively for decision-making. The absence of spatial visualization tools limits the potential of this valuable data to inform critical interventions for sustainable development. Recognizing these challenges, this project, **PeyyumMazha: GIS-Based Rainfall Mapping System**, has been developed to automate the transformation of raw rainfall data into intuitive, spatially distributed rainfall maps using Geographic Information Systems (GIS) and Python-based open-source technologies. By leveraging data visualization techniques and GIS frameworks, the system aims to enhance the interpretability of rainfall patterns over geographic regions, enabling actionable insights for planners, disaster management authorities, researchers, and the farming community. The system also aligns with global sustainability efforts by contributing to the United Nations Sustainable Development Goals (SDGs), particularly SDG 2 (Zero Hunger), SDG 6 (Clean Water and Sanitation), SDG 11 (Sustainable Cities and Communities), and SDG 13 (Climate Action). The introduction sets the foundation for understanding the challenges, objectives, and the significant role of GIS in addressing the need for effective rainfall visualization, outlining how the system integrates engineering principles to provide a practical, scalable solution for rainfall monitoring and analysis.

1.2 OBJECTIVE:

The primary objective of the PeyyumMazha GIS-Based Rainfall Mapping System is to develop an automated, scalable tool that generates spatially distributed rainfall maps using rainfall data collected from reliable sources. The system aims to transform raw numerical rainfall data into intuitive, color-coded maps using Python and GIS technologies, making rainfall patterns easier to interpret and utilize for planning purposes. Additionally, the system seeks to facilitate spatial analysis of rainfall variability across different regions, enabling the identification of high and low rainfall zones, supporting agricultural planning, flood monitoring, and water resource management efforts. By providing an accessible and systematic rainfall visualization tool, the system aims to assist stakeholders, including farmers, disaster management authorities, and researchers, in making data-driven decisions for sustainable development and climate adaptation initiatives.

1.3 SCOPE:

The scope of the PeyyumMazha GIS-Based Rainfall Mapping System includes the collection and processing of rainfall data with geographic coordinates, generating gridded rainfall maps for the state of Kerala using open-source GIS frameworks. The system will handle rainfall data ingestion, spatial grid generation, interpolation, visualization, and export of high-quality rainfall distribution maps. The project focuses on generating static rainfall maps, which can later be integrated into dashboards or reporting systems for operational monitoring. While the current

system focuses on rainfall mapping, it lays the groundwork for future integration of real-time rainfall monitoring, anomaly detection, and additional hydrological layers for comprehensive environmental analysis. The system will serve as a practical tool for researchers, planners, and agencies working in the domains of agriculture, disaster management, and water resources, aligning with the objectives of sustainable and climate-resilient development.

1.4 MOTIVATION:

The motivation for developing the PeyyumMazha GIS-Based Rainfall Mapping System arises from the need to enhance the utility of rainfall data for planning and decision-making in rainfall-dependent regions. Despite the availability of rainfall data from meteorological stations, the absence of spatial visualization limits its effective use in monitoring rainfall variability, predicting flood-prone zones, and managing water resources efficiently. The challenges faced by farmers in planning irrigation, authorities in managing flood risks, and researchers in analyzing climatic patterns emphasize the necessity of transforming rainfall data into a format that is easily understandable and actionable. GIS offers powerful capabilities for spatial data visualization, allowing the integration of rainfall data with geographic information to generate detailed maps that can support effective decision-making. This project aims to bridge the gap between rainfall data collection and actionable insights by providing an automated, scalable system for rainfall mapping, driven by the vision of empowering communities and planners to utilize rainfall information effectively for sustainable development and disaster resilience.

CHAPTER 2

TECHNICAL APPROACH

The PeyyumMazha GIS-Based Rainfall Mapping System adopts a systematic, structured technical approach to convert rainfall data into visually interpretable spatial maps through Python programming and GIS frameworks. The process begins with **data ingestion**, where rainfall data containing geographic coordinates (latitude and longitude) and rainfall measurements are collected from reliable CSV or Excel sources. This ensures that each rainfall data point is georeferenced accurately, enabling precise placement on the geographic grid during map generation. The next stage involves **grid generation** using numpy to create a structured mesh grid over the region of interest, allowing uniform sampling of the spatial domain and facilitating accurate interpolation of rainfall values across unsampled locations. Following grid generation, **spatial interpolation** is performed using the `scipy.interpolate.griddata` method, employing linear or cubic interpolation to estimate rainfall intensities at each grid cell based on surrounding data points, thereby creating a smooth and continuous rainfall distribution surface across the mapped region.

After interpolation, the system advances to the **visualization stage**, utilizing matplotlib and cartopy to overlay the interpolated rainfall data on the Kerala basemap, including district boundaries and coastlines for contextual understanding. The visualization is further enhanced using color maps that represent different rainfall intensities, making it easier for stakeholders to distinguish between heavy, moderate, and light rainfall zones. Finally, the system includes a **map exporting module**, where the generated rainfall maps are saved in high-resolution image formats such as PNG or JPEG, enabling their use in reports, presentations, research analysis, and integration with other monitoring tools. This systematic approach ensures that the rainfall mapping process is automated, accurate, and scalable while maintaining flexibility for adjustments in grid resolution, color schemes, and input data, making it a valuable engineering solution for rainfall monitoring.

CHAPTER 3

SYSTEM ARCHITECTURE

The architecture of the PeyyumMazha GIS-Based Rainfall Mapping System is designed to ensure modularity, clarity, and scalability, allowing systematic handling of rainfall data from ingestion to map generation. The system begins with an **Input Module** responsible for loading rainfall data from CSV files, ensuring data types are validated and handling missing values where necessary to maintain data integrity. This module also ensures that the geographic boundaries of the data align with the region of interest, which, in this case, is Kerala. The **Grid Generation Module** uses the numpy library to generate a mesh grid with specified resolution over the geographic extent, ensuring coverage of all rainfall data points while allowing the flexibility to adjust the grid size for higher resolution or faster processing based on the analysis needs.

The **Interpolation Module** applies `scipy.interpolate.griddata` to estimate rainfall values across each grid point using the loaded rainfall data, ensuring seamless interpolation while maintaining the accuracy of original measurements. The **Visualization Module** employs matplotlib and cartopy to plot the interpolated rainfall data over Kerala's basemap, customizing color gradients to reflect rainfall intensities effectively, adding coastlines, administrative boundaries, and legends for clear interpretability. The **Export Module** saves these generated maps to the local system, ready for reporting and further analysis. The workflow follows a clear, logical sequence, starting from rainfall data ingestion and cleaning, followed by grid generation, interpolation, visualization, and finally exporting the output maps for stakeholders.

This architecture not only simplifies the implementation but also enhances the maintainability of the system by isolating functionalities into distinct modules, enabling easy debugging, upgrading, and future integration with automated pipelines or web-based dashboards. It is designed to handle batch data processing, ensuring the system is capable of generating rainfall maps on a daily, weekly, or monthly basis, aligning with operational monitoring requirements for government agencies and research institutions.

CHAPTER 4

TECHNICAL MODULES IMPLEMENTED

The **PeyyumMazha system** is architected with **multiple modular components**, each meticulously designed to ensure that the system remains functional, scalable, reliable, and adaptable to future enhancements while simplifying the rainfall mapping workflow for practical use in decision-making environments.

The first component, the **Data Collection and Cleaning Module**, is responsible for systematically ingesting rainfall data from external CSV or Excel files sourced from meteorological departments or rain gauge networks. This module verifies the consistency of data types by ensuring that numerical columns such as latitude, longitude, and rainfall measurements are correctly formatted, which is essential for accurate downstream processing. The module is equipped with functions to detect and handle missing or erroneous data entries, employing strategies such as interpolation or removal of invalid rows, thereby preparing a clean, structured dataset for analysis. This ensures that the system maintains high data integrity, providing a reliable foundation for subsequent mapping processes.

Following data preparation, the **Grid Generation Module** defines the spatial framework over which rainfall data will be visualized. Using Python's numpy library, this module generates structured arrays of latitude and longitude values, determining the spatial resolution and extent of the grid that will cover the region of Kerala. The resolution of this grid is customizable, enabling the system to balance the trade-off between computational efficiency and the granularity of rainfall details presented in the maps. By structuring the spatial domain into a mesh grid, the system ensures consistent reference points across the region for effective rainfall interpolation.

The **Interpolation Module** is a critical component that transforms discrete rainfall data points into a continuous rainfall distribution map. Leveraging the robust algorithms provided by the scipy.interpolate library, this module performs spatial interpolation using methods such as linear or cubic interpolation to estimate rainfall values across unsampled grid cells based on nearby data points. This process effectively fills the spatial gaps in the dataset, producing a smooth and continuous rainfall surface that accurately reflects the variability in rainfall patterns across Kerala while preserving the precision of the input measurements.

Visualization plays a central role in making rainfall data interpretable, and the **Visualization Module** is designed to fulfill this need efficiently. Utilizing matplotlib and cartopy, this module overlays the interpolated rainfall data on geographic basemaps of Kerala, applying visually appealing and intuitive color schemes to represent varying rainfall intensities. The color scales are carefully chosen to indicate low, moderate, and high rainfall zones clearly, making it easy for users to interpret rainfall patterns at a glance. Additionally, the module enhances contextual clarity by adding layers such as district boundaries, coastlines, and relevant geographic features, ensuring that the maps generated are not only visually attractive but also informative for practical decision-making in agricultural planning and disaster preparedness.

The final step in the pipeline is managed by the **Map Export Module**, which is tasked with saving the generated rainfall maps in high-quality formats such as PNG or JPEG. This enables seamless inclusion of the maps in decision-support systems, government and research reports,

presentations, and stakeholder communication platforms. By providing high-resolution outputs, the module ensures that the visual details and color integrity are preserved for analysis and reporting, making the outputs ready for immediate practical use.

Each of these modules in the PeyyumMazha system is designed to be **modular, reusable, and easily maintainable**, allowing developers or stakeholders to debug, upgrade, or expand specific functionalities without disrupting the entire system. This modular approach enables future enhancements, including the integration of real-time rainfall data streams through APIs, advanced thematic map styling with custom legends and overlays for specialized applications, and the incorporation of automated scheduling using cron jobs or cloud functions for continuous and real-time rainfall monitoring. By structuring the system in this modular fashion, PeyyumMazha not only addresses the current challenges in rainfall data visualization but also establishes a scalable framework capable of evolving into a comprehensive rainfall monitoring and climate resilience tool.

CHAPTER 5

IMPLEMENTATION

5.1 Rainfall Data Collection via Whatsapp

1	6/24/25, 10:53 AM - Messages and calls are end-to-end encrypted. Only people in this chat can read, listen to, or share them. Learn more.
2	6/6/24, 4:16 PM - +91 80869 45547 created group "CLIMATE-BALUSSERY BLOCK"
3	6/24/25, 10:53 AM - ~ Kerala Sasthrasahithya Parishad Balussery added you
4	6/24/25, 12:12 PM - +91 70124 46771: 24/6/25
5	എറമംഗലം
6	32 mm
7	6/24/25, 3:20 PM - +91 80780 39635: <Media omitted>
8	6/25/25, 5:24 AM - +91 90371 13122: തിയ്യതി :25.06.2025
9	
10	സ്ഥലം :ബാലുശ്ശേരി ബ്ലോക്ക് റേഡ്
11	
12	മഴയുടെ അളവ് : 53 MM
13	6/25/25, 5:47 AM - +91 98474 60783: 25.6.25
14	കൊയക്കുട്.
15	42.mm
16	6/25/25, 6:01 AM - +91 99460 57427: 25.06.2025.. തൃക്കുറ്റിമുളി...51..3.MM
17	6/25/25, 7:01 AM - +91 99468 21267: Ekarool vallyoth60.2mm
18	6/25/25, 7:02 AM - +91 86061 74843: 25.6.25 നിർമ്മലുർ 46mm
19	6/25/25, 7:07 AM - +91 94964 24932: 25-6-25
20	Koottalida
21	63.2mm
22	Minimum - 24
23	Maximum - 29
24	6/25/25, 7:16 AM - +91 94463 76166: 25/6/25
25	Kunnikkottam
26	52.6mm
27	6/25/25, 7:22 AM - +91 94974 04272: 25/06/25
28	Kakkayam
29	72mm
30	6/25/25, 7:49 AM - +91 99463 79530: കുന്നപ്ലാറ്റം 38mm
31	6/25/25, 7:59 AM - +91 94460 52978: 25-6- 25 അപ്പിടിക 49.8 mm
32	6/25/25, 8:07 AM - +91 86060 50510: 25.06.2025
33	PUTHANCHERY
34	മഴയുടെ അളവ്:33.6 mm
35	6/25/25, 8:09 AM - +91 90488 81520: 25/6/25
36	തേന്തകുഴി 52mm
37	6/25/25, 8:12 AM - +91 94478 76687: 25.06.2025 *മുലാട്*
38	
39	കൂടിയ താപനില 27.5°C
40	കുറഞ്ഞ താപനില 23°C
41	6/25/25, 8:12 AM - +91 94958 59871: 25 -6- 25
42	Ulliyeri
43	Rain : 48.2 mm
44	
45	Temp
46	Mini : 23.5°C
47	Max: 28 °C
48	6/25/25, 8:13 AM - +91 80864 01378: *Date: 25-06-2025*
49	
50	*Place: Vayalada*
51	
52	*Rainfall Measured: 57 mm*
53	*_____*
54	*Minimum Temperature: 21° C*
55	
56	*Maximum Temperature: 27°C*
57	6/25/25, 8:14 AM - +91 94460 94092: *SIVAPURAM*
58	25.06 .2025
59	മഴ 40.8mm
60	കൂടിയ താപനില 27

61	കുറഞ്ഞ താപനില 24
62	6/25/25, 8:15 AM - +91 97470 75323: 25.06.25
63	കോക്കല്ലൂർ
64	52.3mm
65	6/25/25, 8:16 AM - +91 94969 18373: മണിച്ചേരി,
66	
67	25/06/25 (ബുധൻ)
68	
69	മഴയളവ് : 55.4 mm
70	6/25/25, 8:41 AM - +91 98474 25358: 25. 06.25
71	മണ്ണാമ്പൊയിൽ
72	മഴയളവ്
73	43. 4 mm
74	6/25/25, 8:49 AM - +91 89077 07751: 25/6/25
75	നടുവണ്ണൂർ
76	50.2mm
77	കൂടിയ താപനില 30°C
78	കുറഞ്ഞ താപനില 23°C
79	6/25/25, 9:07 AM - +91 70124 46771: 25/6/25
80	എരമംഗലം
81	42.3 mm
82	6/25/25, 9:55 AM - +91 96560 19389: 25-06-25
83	ചീടിക്കുഴി
84	64.2 mm
85	6/25/25, 10:23 AM - +91 99468 15038: Kinalur
86	Rain 49.8mm
87	Min 24
88	Max 28
89	6/25/25, 10:29 AM - +91 95395 82514: 25 .06 .25
90	പനങ്ങാട്
91	59.8mm
92	6/25/25, 10:32 AM - +91 99463 37150: 25/6/2025
93	Vakayad
94	38mm
95	6/25/25, 11:42 AM - +91 62352 37141: <Media omitted>
96	6/25/25, 11:53 AM - +91 98469 36171: നിലവിൽ ആശങ്ക ഒന്നും തന്നെ ഇല്ല
97	6/25/25, 1:17 PM - +91 99470 38323: **സ്ഥലം :ഇയാട്, മഴയുടെ അളവ് :- 70.20 MM .:* രാവിലത്തെ കണക്കാണ് (നെറ്റ് പ്രൊജക്ട്)
98	6/25/25, 1:31 PM - +91 80780 39635: <Media omitted>
99	6/25/25, 4:47 PM - +91 98469 36171: മഴ കൂടുന്ന സാഹചര്യത്തിൽ നിങ്ങളുടെ പ്രദേശത്ത് നിരവധിയിൽ ഏതെങ്കിലും തരത്തിലുള്ള നിറവുത്യാസം, വിള്ളലുകൾ
100	
101	ഒപ്പം തന്നെ മഴയുടെ അളവുകൾ കൃത്യമായി ഗൂഗിൾ അപ്ഡേറ്റ് ചെയ്താൽ മാത്രമേ ആ പ്രദേശത്തെ മഴയുടെ മാതൃക മനസ്സിലാക്കാനും വരും ദിവസങ്ങളിലെ
102	6/25/25, 5:18 PM - +91 62352 37141: <Media omitted>
103	6/26/25, 5:25 AM - +91 90371 13122: തിയ്യതി :26.06.2025
104	
105	സ്ഥലം :ബാലുശ്ശേരി ബ്ലോക്ക് റോഡ്
106	
107	മഴയുടെ അളവ് : 46 MM
108	6/26/25, 5:57 AM - +91 98474 60783: കൊയക്കോട്
109	19. mm
110	6/26/25, 6:04 AM - +91 99470 38323: **സ്ഥലം :ഇയാട്, മഴയുടെ അളവ് :- 50.10 MM .:*
111	6/26/25, 6:09 AM - +91 99460 57427: 26.06.2025..തൃക്കുറ്റിശ്ശേരി.. 26..1.MM
112	6/26/25, 6:16 AM - +91 99468 21267: Ekaroot vallyoth48.4mm
113	6/26/25, 6:41 AM - +91 86061 74843: 26.6.25നിർമ്മലൂർ 44mm
114	6/26/25, 7:01 AM - +91 94460 52978: 26-6-25 അറപ്പിടിക 45 .2 mm
115	6/26/25, 7:05 AM - +91 95395 82514: 26 .06 .25
116	പനങ്ങാട്
117	47.5mm
118	6/26/25, 7:07 AM - +91 94463 76166: 26/6/25
119	Kunnikkootam
120	33.4mm
121	6/26/25, 7:15 AM - +91 98474 25358: 26. 06.25
122	മണ്ണാമ്പൊയിൽ
123	മഴയളവ്
124	27. 3 mm
125	6/26/25, 7:21 AM - +91 96560 19389: 26 - 06 - 25
126	ചീടിക്കുഴി
127	80.3 mm
128	6/26/25, 7:50 AM - +91 94969 18373: മണിച്ചേരി ,
129	
130	26/06/25 (വ്യാഴം)
131	
132	മഴയളവ്: 62 mm
133	6/26/25, 8:00 AM - +91 94957 90106: 26/6/25
134	,chavittanpara
135	75mm
136	
137	28°C
138	24°C
139	6/26/25, 8:01 AM - +91 94964 24932: 26-6-25
140	Koottalida
141	52.mm
142	Minimum - 25
143	Maximum - 28
144	6/26/25, 8:04 AM - +91 94478 76687: 26.06.2025 *മുലാട്*
145	
146	കൂടിയ താപനില 27°C
147	കുറഞ്ഞ താപനില 24°C

148 6/26/25, 8:11 AM - +91 90488 81520: 26/6/25
 149 മേന്മാകുഴി 48mm
 150 6/26/25, 8:11 AM - +91 94460 94092: *SIVAPURAM*
 151 26.06 .2025
 152 മഴ 50 mm
 153 കൂടിയ താപനില 26.5
 154 കുറഞ്ഞ താപനില 24
 155 6/26/25, 8:16 AM - +91 99468 15038: Kinalur
 156 Rain 58mm
 157 Min 25
 158 Max 27
 159 6/26/25, 8:20 AM - +91 94974 04272: 26/06/25
 160 Kakkayam
 161 84mm
 162 6/26/25, 8:23 AM - +91 94958 59871: 26-6- 25
 163 Ulliyer1
 164 Rain :25 mm
 165
 166 Temp
 167 Mini:24 °C
 168 Max: 28 °C
 169 6/26/25, 8:34 AM - +91 89077 07751: 26/6/25
 170 നടുവണ്ണൂർ
 171 43mm
 172 കൂടിയ താപനില28°C
 173 കുറഞ്ഞ താപനില 24°C
 174 6/26/25, 8:40 AM - +91 97470 75323: 26.06.25
 175 കോക്കല്ലൂർ
 176 45.1mm
 177 6/26/25, 8:48 AM - +91 99463 37150: 26/6/2025
 178 Vakayad
 179 41mm
 180 6/26/25, 9:49 AM - +91 80780 39635: *മഴ കനക്കുന്ന സാഹചര്യത്തിൽ മഴയുടെ അളവ് കൃത്യസമയത്തു തന്നെ ഗ്രൂപ്പിൽ അറിയിക്കുമല്ലോ* <This mes
 181 6/26/25, 9:49 AM - ~ രഞ്ജിനി pinned a message

Fig 5.1 Rainfall data collection via Whastapp

5.2 Climate Log

1	Date	Place	rainfall_mm
2	2025-06-2	Eramangal	32
3	2025-06-2	Balusser	53
4	2025-06-2	Kootalida	63.2
5	2025-06-2	Kunnikkoo	52.6
6	2025-06-2	Kakkayam	72
7	2025-06-2	Puthanche	33.6
8	2025-06-2	Ulliyeri	48.2
9	2025-06-2	Vayalada	57
10	2025-06-2	Highly Tem	40.8
11	2025-06-2	Kokkallur	52.3
12	2025-06-2	250625 W	55.4
13	2025-06-2	Punjalur	50.2
14	2025-06-2	Eramangal	42.3
15	2025-06-2	Cheedikku	64.2
16	2025-06-2	Panangad	59.8
17	2025-06-2	Balusser	46
18	2025-06-2	Panangad	47.5
19	2025-06-2	Kunnikkoo	33.4
20	2025-06-2	Cheedikku	80.3
21	2025-06-2	260625 Th	62
22	2025-06-2	Chavittan	75
23	2025-06-2	265 Pm	50
24	2025-06-2	Kakkayam	84
25	2025-06-2	Ulliyeri	25
26	2025-06-2	Punjalur	43
27	2025-06-2	Kokkallur	45.1
28	2025-06-2	Vayalada	41

Fig 5.2 Climate Log

5.3 Python Script for Structuring Whatsapp Rainfall Data

```
94 import re
95 import os
96 import pandas as pd
97 from deep_translator import GoogleTranslator
98 from datetime import datetime
99 from dateutil import parser
100
101 # Translator
102 translator = GoogleTranslator(source='auto', target='en')
103
104 # Output path
105 output_dir = "."
106 os.makedirs(output_dir, exist_ok=True)
107
108 # Read WhatsApp chat
109 with open("chat.txt", "r", encoding="utf-8") as f:
110     lines = [line.strip() for line in f if line.strip()]
111
112 # Data and variables
113 data = []
114 current_date = None
115 rainfall = None
116 place = None
117
118 manual_place_fixes = {
119     "Eramangalam": "Eramangalam",
120     "Balusseri": "Balussery",
121     "Location Balussery Block Road": "Balussery",
122     "Thrikurissheeri": "Thrikkurissery",
123     "Kakkayam": "Kakkayam",
124     "Kakkayam": "Kakkayam",
125     "Cokalore": "Kokkallur",
126     "Yakkay": "Kakkayam",
127     "Panegadu": "Panangad",
128     "Municipal Mate": "Municipality",
129     "Vakayad": "Vayalada",
130     "Koottalida": "Kootalida",
131     "Kunnikkottathil": "Kunnikkootathil",
132     "Place Vayalada": "Vayalada",
133     "Kokkallur": "Kokkallur",
134     "Kunnikkootam": "Kunnikkootathil",
135     "Cloudy": "Cheedikkuzhi"
136 }
137
138 # Patterns
139 date_pattern = re.compile(r'(\d{1,2}[/\.\-])\d{1,2}[/\.\-])\d{2,4}')
140 rainfall_pattern = re.compile(r'(\d{1,3})(?:[.]\d{1,2})?\s*mm', re.IGNORECASE)
141
142 # Process each line and look around it
143 for i in range(len(lines)):
144     line = lines[i].replace('\u202f', ' ').replace(' ', ' ').strip()
145
146     # Skip obvious non-data lines
147     if "min" in line.lower() or "max" in line.lower() or "temperature" in line.lower():
148         continue
149
150     # 1. Extract rainfall
151     rain_match = rainfall_pattern.search(line)
152     if rain_match:
```

```

152         try:
153             rainfall = float(rain_match.group(1).replace(",", "."))
154         except:
155             rainfall = None
156
157         # 2. Try to find date from nearby lines
158         current_date = None
159         for j in range(i, max(i - 3, -1), -1):
160             date_match = date_pattern.search(lines[j])
161             if date_match:
162                 try:
163                     current_date = parser.parse(date_match.group(1), dayfirst=True).strftime("%Y-%m-%d")
164                     break
165                 except:
166                     pass
167
168         # 3. Try to find place from nearby lines
169         place = None
170         for j in [i - 1, i + 1]: # look before/after
171             if 0 <= j < len(lines):
172                 pline = lines[j].strip()
173                 if not re.search(r'(\d{1,2}[:.]?\d{2})', pline, re.IGNORECASE) and not pline.endswith(("AM", "PM")):
174                     place = pline.strip(".,:-/")
175                     break
176
177         if current_date and place and rainfall is not None:
178             try:
179                 # Translate Malayalam if needed
180                 if any("\u0000" <= c <= "\u007F" for c in place):
181                     place = translator.translate(place).strip().title()
182             except:
183                 place = place.strip().title()
184
185             # Clean unicode/emoji noise
186             place = re.sub(r"[\u0000-\u007F]", "", place).title()
187
188             # Apply manual fixes
189             place = manual_place_fixes.get(place, place)
190         except Exception as e:
191             print(f"⚠️ Translation error: {e}")
192             continue
193
194         data.append([current_date, place, rainfall])
195         print(f"✅ {current_date} | {place} | {rainfall} mm")
196
197         # Reset rainfall
198         rainfall = None
199
200     # Save results
201     if data:
202         df = pd.DataFrame(data, columns=["Date", "Place", "Rainfall_mm"])
203         df.drop_duplicates(subset=["Date", "Place"], inplace=True)
204         output_path = os.path.join(output_dir, "climate_log.xlsx")
205         df.to_excel(output_path, index=False)
206         print(f"✅ Data saved to {output_path}")
207     else:
208         print("⚠️ No valid rainfall entries found.")

```

Fig 5.3 Python Script for Structuring Whatsapp Rainfall Data

5.4 Python GIS Rainfall Mapping

```
69 import os
70 import pandas as pd
71 import geopandas as gpd
72 import matplotlib.pyplot as plt
73 import matplotlib.path_effects as path_effects
74 from matplotlib.patches import Patch
75
76 # Ensure maps folder exists
77 os.makedirs("maps", exist_ok=True)
78
79 # File paths
80 climate_data_path = "climate_log.xlsx"
81 grid_mapping_path = "grid_mapping.xlsx"
82 shapefile_path = "balusseri_grid/BALUSSERY_GRID.shp"
83
84 # Load data
85 print("📁 Loading climate and mapping data...")
86 climate_df = pd.read_excel(climate_data_path)
87 mapping_df = pd.read_excel(grid_mapping_path)
88 gdf = gpd.read_file(shapefile_path)
89
90 # Get latest date and construct output map filename
91 latest_date = climate_df["Date"].max()
92 output_map_path = f"maps/map_{latest_date}.png"
93 print(f"📅 Latest available date: {latest_date}")
94
95 # Filter to latest date
96 latest_rainfall = climate_df[climate_df["Date"] == latest_date]
97
98 # Merge rainfall data with GRID_IDs
99 merged_df = pd.merge(latest_rainfall, mapping_df, on="Place", how="left")
100
101 # Merge with shapefile GeoDataFrame
102 gdf = gdf.merge(merged_df, on="GRID_ID", how="left")
103
104 # Color assignment function
105 def get_color(rainfall):
106     if pd.isna(rainfall):
107         return "#f0f0f0" # very light gray
108     elif rainfall <= 10:
109         return "#b3e2cd" # soft mint green
110     elif rainfall <= 30:
111         return "#a6dcef" # soft sky blue
112     elif rainfall <= 60:
113         return "#7aee3" # soft cornflower blue
114     else:
115         return "#c49bcf" # soft lavender purple
116
117 # Assign colors
118 gdf["color"] = gdf["Rainfall_mm"].apply(get_color)
119
120 # Plotting
121 fig, ax = plt.subplots(figsize=(12, 12))
122 gdf.plot(ax=ax, color=gdf["color"], edgecolor="black")
123
124 # Add labels
125 for _, row in gdf.iterrows():
126     if pd.isna(row["Rainfall_mm"]) or row["geometry"].centroid.is_empty:
127         continue
128     x, y = row["geometry"].centroid.coords[0]
```

```

129     label = f"{row['Place']}\n{int(row['Rainfall_mm'])} mm"
130     text = ax.text(x, y, label, ha='center', fontsize=6, color='white')
131     text.set_path_effects([
132         path_effects.Stroke(linewidth=1.5, foreground='black'),
133         path_effects.Normal()
134     ])
135
136     # Add legend
137     legend_elements = [
138         Patch(facecolor="#b3e2cd", edgecolor='black', label="0-10 mm"),
139         Patch(facecolor="#a6dcef", edgecolor='black', label="11-30 mm"),
140         Patch(facecolor="#7aaee3", edgecolor='black', label="31-60 mm"),
141         Patch(facecolor="#c49bcf", edgecolor='black', label="≥60 mm"),
142         Patch(facecolor="#f0f0f0", edgecolor='black', label="No Data")
143     ]
144     ax.legend(handles=legend_elements, title="Rainfall (mm)", loc='lower center',
145             bbox_to_anchor=(0.5, -0.1), ncol=5, frameon=False)
146
147     # Final touches
148     ax.set_title(f"Rainfall Map - {latest_date}", fontsize=16)
149     ax.axis("off")
150     plt.tight_layout()
151
152     # Save the map
153     plt.savefig(output_map_path, dpi=300)
154     print(f"📍 Rainfall grid map saved to {output_map_path}")

```

Fig 5.4 Python GIS Rainfall Mapping

5.5 Grid Mapping

1	Place	GRID_ID
2	Mandhankavu	I-3
3	Kakkancheri	J-3
4	Arapeedika	I-4
5	Balusseri	K-3
6	Panangad	J-4
7	Nirmalloor	D-4
8	Kokkallur	E-7
9	Ekarool	B-7
10	Ulliyeri	L-4
11	Thenakuzhi	E-9
12	Naduvannur	I-7
13	Kinaloor	B-9
14	Kakkayam	H-10
15	Iyyad	I-8
16	Kunnikkoottathil	D-9
17	Vakayad	E-8
18	Thrikkuttissery	F-8
19	Shivapuram	G-5
20	Mannampoyil	E-6
21	Puthanchery	K-2
22	Eramangalam	I-9
23	Manicheri	H-9
24	Kootalida	D-8
25	Vayalada	G-7
26	Cheedikkuzhi	G-6

Fig 5.5 Grid Mapping

5.6 Rainfall Grid Viewer

```
1  import streamlit as st
2  import pandas as pd
3  import os
4  from datetime import datetime
5  from PIL import Image
6  import re
7  from deep_translator import GoogleTranslator
8  from dateutil import parser
9
10 # Setup
11 st.set_page_config(page_title="Balusseri Rainfall App", layout="centered")
12 st.title("☁ Balusseri Rainfall Mapping App")
13
14 # Paths
15 MAP_DIR = "maps"
16 CHAT_PATH = "app/chat.txt"
17 EXCEL_PATH = "app/climate_log.xlsx"
18
19 # Ensure folders exist
20 os.makedirs(MAP_DIR, exist_ok=True)
21 os.makedirs("app", exist_ok=True)
22
23 # Function to extract rainfall data from chat
24 def extract_climate_data(chat_path):
25     with open(chat_path, "r", encoding="utf-8") as f:
26         lines = f.readlines()
27
28     translator = GoogleTranslator(source='auto', target='en')
29     manual_place_fixes = {
30         "Erramangalam": "Erramangalam",
31         "Balusseri": "Balussery",
32         "Balusseri": "Balussery",
33         "Location Balussery Block Road": "Balussery",
34         "Thrikurisscheri": "Thrikurissery",
35         "Kakkayam": "Kakkayam",
36         "Cokalore": "Kokkallur",
37         "Yakkay": "Kakkayam",
38         "Panegadu": "Panangad",
39         "Municipal Mate": "Municipality",
40         "Vakayad": "Vayalada",
41         "Koottalida": "Kootalida",
42         "Kunnikkottathil": "Kunnikkootathil",
43         "Place Vayalada": "Vayalada",
44         "Kokkallur": "Kokkallur",
45         "Kunnikkootam": "Kunnikkootathil",
46         "Cloudy": "Cheedikkuzhi"
47     }
48
49     data = []
50     current_date = None
51
52     for i, line in enumerate(lines):
53         line = line.strip()
54
55         # WhatsApp timestamp line
56         date_msg = re.match(r"^(?!\d{1,2}/\d{1,2}/\d{2,4}),\s+(?!\d{1,2}:\d{2})", line)
57         if date_msg:
58             try:
59                 current_date = parser.parse(date_msg.group(1), dayfirst=True).strftime("%Y-%m-%d")
60             except:
```

```

60         continue
61
62     # Explicit date line
63     explicit_date = re.search(r"\b\d{1,2}/\d{1,2}/\d{2,4}\b", line)
64     if explicit_date:
65         try:
66             current_date = parser.parse(explicit_date.group(), dayfirst=True).strftime("%Y-%m-%d")
67             continue
68         except:
69             pass
70
71     # Format: "Place 32 mm"
72     combined = re.match(r"^(.+?)\s+(\d+)\s*mm$", line, re.IGNORECASE)
73     if combined:
74         place_line = combined.group(1).strip()
75         rainfall = int(combined.group(2))
76         try:
77             translated_place = translator.translate(place_line).strip()
78             translated_place = manual_place_fixes.get(translated_place, translated_place)
79         except:
80             translated_place = place_line
81         data.append([current_date, translated_place, rainfall])
82         continue
83
84     # Separate format: place above, rainfall below
85     if re.search(r"\d+\s*mm", line.lower()):
86         rainfall_match = re.search(r"(\d+)\s*mm", line.lower())
87         if rainfall_match:
88             rainfall = int(rainfall_match.group(1))
89             place_line = lines[i - 1].strip() if i > 0 else "Unknown"
90             try:
91                 translated_place = translator.translate(place_line).strip()
92                 translated_place = manual_place_fixes.get(translated_place, translated_place)
93             except:
94                 translated_place = place_line
95             data.append([current_date, translated_place, rainfall])
96
97     df = pd.DataFrame(data, columns=["Date", "Place", "Rainfall_mm"])
98     df.to_excel(EXCEL_PATH, index=False)
99     return df
100
101 # Upload chat
102 st.header("📁 Upload WhatsApp Chat File")
103 uploaded_file = st.file_uploader("Upload WhatsApp chat.txt", type=["txt"])
104
105 if uploaded_file:
106     with open(CHAT_PATH, "wb") as f:
107         f.write(uploaded_file.getvalue())
108     st.success("✅ Chat uploaded successfully.")
109
110     with st.spinner("🔄 Extracting rainfall data..."):
111         df = extract_climate_data(CHAT_PATH)
112
113     st.success("✅ Rainfall data extracted and saved!")
114     st.dataframe(df.tail())
115

```



```

116     # Download extracted Excel
117     st.download_button(
118         label="📄 Download Extracted Excel",
119         data=open(EXCEL_PATH, "rb").read(),
120         file_name="climate_log.xlsx",
121         mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
122     )
123
124     # View generated maps
125     st.header("🗺️ View Rainfall Map")
126
127     map_files = sorted([
128         f for f in os.listdir(MAP_DIR)
129         if f.startswith("map_") and f.endswith(".png")
130     ], reverse=True)
131
132     available_dates = [f.replace("map_", "").replace(".png", "") for f in map_files]
133
134     if not available_dates:
135         st.info("📄 No rainfall maps available yet.")
136     else:
137         selected_date = st.selectbox("Select a date to view the map:", available_dates)
138         selected_path = os.path.join(MAP_DIR, f"map_{selected_date}.png")
139
140         # Show map
141         st.image(image.open(selected_path), use_column_width=True, caption=f"Rainfall Map - {selected_date}")
142
143         # Add download button for the map
144         with open(selected_path, "rb") as img_file:
145             st.download_button(
146                 label="📄 Download PNG Map",
147                 data=img_file,
148                 file_name=f"Rainfall_Map_{selected_date}.png",
149                 mime="image/png"
150             )
151
152         # Legend
153         with st.expander("📖 Legend (Rainfall in mm)", expanded=False):
154             st.markdown("""
155 - <span style='color:#b3e2cd'>■</span> 0-10 mm
156 - <span style='color:#a6dcef'>■</span> 11-30 mm
157 - <span style='color:#7aee3'>■</span> 31-60 mm
158 - <span style='color:#e49bcf'>■</span> >60 mm
159 - <span style='color:#f0f0f0'>■</span> No Data
160             """, unsafe_allow_html=True)
161
162         # Footer
163         st.markdown("----")
164         st.markdown("<center><sub>Made with for Balusseri by your rainfall mapping app</sub></center>", unsafe_allow_html=True)

```

Fig 5.6 Rainfall Grid Viewer

5.7 Requirements

```
1    pandas
2    geopandas
3    matplotlib
4    streamlit
5    openpyxl
6    googletrans==4.0.0-rc1
7    python-dateutil
8    fiona
9    shapely
10   pyproj
11   rtree
```

Fig 5.7 Requirements

5.8 run_mapping_pipeline.bat

```
1    @echo off
2    cd app
3
4    echo Running extract_climate.py...
5    python extract_climate.py
6
7    echo Running generate_grid_map.py...
8    python generate_grid_map.py
9
10   echo Done. Press any key to exit...
11   pause
```

Fig 5.8 run_mapping_pipeline.bat

CHAPTER 6

RESULT AND DISCUSSION

The PeyyumMazha GIS-Based Rainfall Mapping System was implemented to generate clear, spatially distributed rainfall maps for the Kerala region using the developed Python-based GIS pipeline. The system successfully processed rainfall data collected from CSV files containing geographic coordinates and rainfall values, performing data cleaning and interpolation to generate continuous rainfall surfaces. The resulting rainfall maps visually depict the distribution of rainfall across the state, highlighting areas with high, moderate, and low rainfall intensities using intuitive color schemes.

The generated maps clearly show spatial patterns that align with the known geographic and climatic characteristics of Kerala, such as higher rainfall along the Western Ghats and lower rainfall in interior regions during certain monsoon phases. The use of `scipy.interpolate.griddata` for interpolation ensured smooth transitions between rainfall values while retaining the accuracy of point-based measurements, and the overlay of district boundaries using `cartopy` added essential context for location-based analysis. The visual outputs were generated in high-resolution PNG and JPEG formats, suitable for direct inclusion in reports and presentations, thereby demonstrating the system's capability as a practical decision-support tool.

The discussion also highlights the operational efficiency of the PeyyumMazha system. By automating the process of data ingestion, grid generation, interpolation, visualization, and export, the system significantly reduces manual intervention and the time required to produce high-quality rainfall maps. This allows for rapid updates, enabling the system to be integrated into daily or weekly rainfall monitoring processes. The modular structure of the system also facilitated easy adjustments to grid resolution, color mapping, and input data, reflecting its flexibility in adapting to different datasets and user requirements.

During testing, the system demonstrated stability and consistency in handling varying data volumes, indicating scalability for larger datasets or for extending the system to other regions with minor adjustments. The results obtained through the PeyyumMazha system confirm its effectiveness in transforming numerical rainfall data into actionable, easy-to-interpret visual insights, aiding planners, researchers, and disaster management authorities in making informed, data-driven decisions for climate resilience, agriculture, and water resource management.

Rainfall Map - 2025-06-26

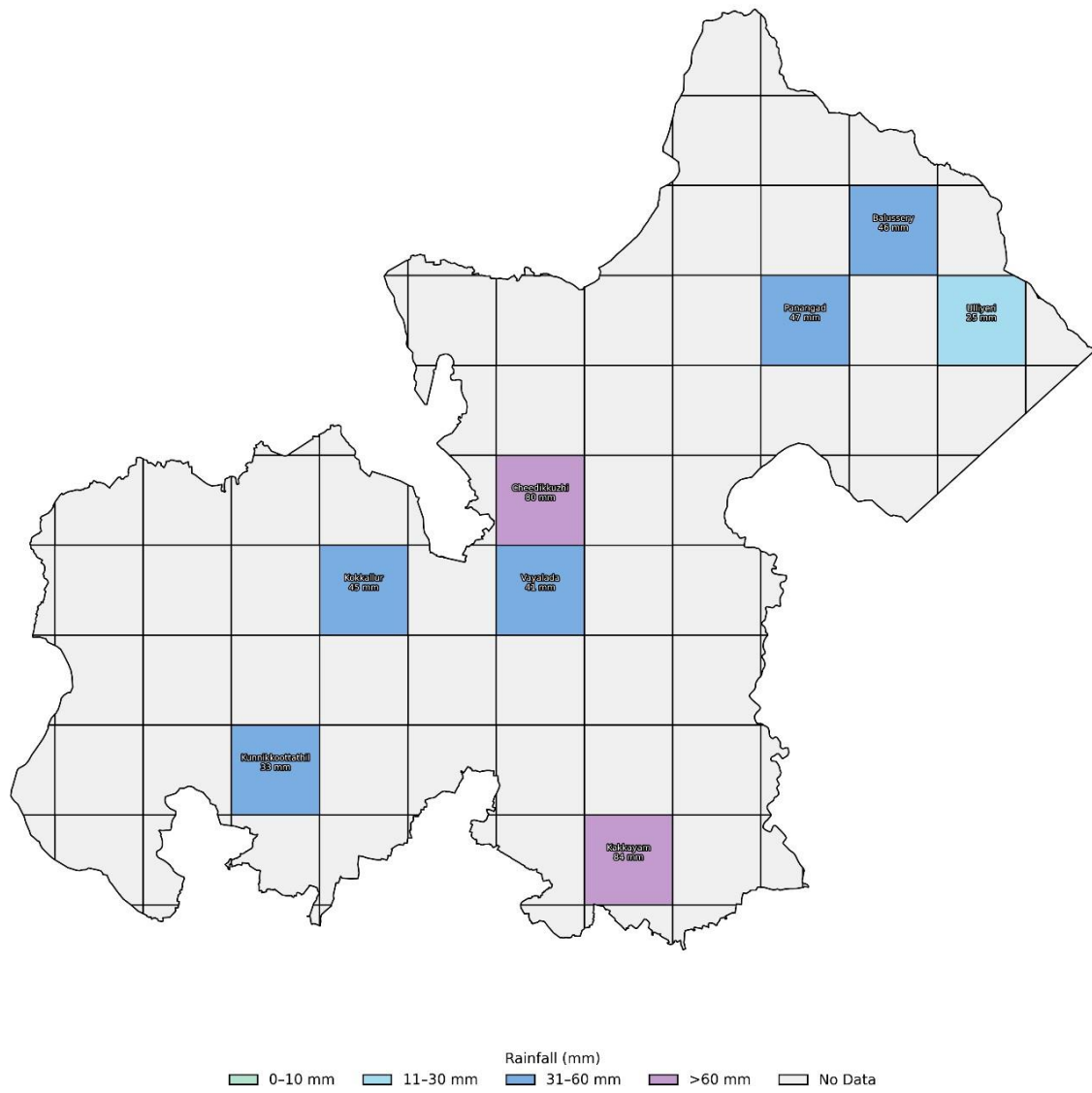


Fig 6.1 Rainfall Map

CHAPTER 7

CONCLUSION

The PeyyumMazha GIS-Based Rainfall Mapping System successfully demonstrates how Python-based GIS tools can transform raw rainfall data into visually interpretable, spatially distributed rainfall maps. By automating the stages of data collection, cleaning, grid generation, interpolation, visualization, and export, the system bridges the gap between numerical data collection and practical, actionable insights for stakeholders in climate-sensitive sectors. The project aligns with the goals of promoting sustainable development and climate resilience by making rainfall data accessible and understandable to decision-makers in agriculture, disaster management, and water resource planning.

The project's systematic workflow ensures repeatability and accuracy in rainfall visualization, enabling planners to identify high and low rainfall regions, understand seasonal rainfall patterns, and prepare effectively for potential flood or drought events. The maps generated by PeyyumMazha not only provide spatial insight but also enhance the interpretability of rainfall data, aiding effective communication among researchers, policy-makers, and communities reliant on timely weather and rainfall information.

Looking ahead, the PeyyumMazha system offers significant scope for enhancement and expansion. Future work could include integrating real-time rainfall data collection using IoT-enabled rain gauges and weather APIs to enable live rainfall monitoring and map updates. Advanced spatial analysis features, such as rainfall anomaly detection and the integration of additional layers like river networks, soil moisture, and flood plains, could extend the system's utility for hydrological modeling and flood risk assessment. Further, developing a web-based GIS dashboard using frameworks like Leaflet or Mapbox can make the system interactive and accessible to a wider range of stakeholders, including government agencies and local communities.

In conclusion, PeyyumMazha represents a scalable, flexible, and practical engineering solution for rainfall monitoring and visualization, contributing towards effective climate adaptation, disaster preparedness, and sustainable agricultural planning, while providing a foundation for future research and system enhancements in the domain of environmental monitoring using GIS technologies.

COMMUNITY WORK RELEVANCE TO NGO

Non-Profit Organizations (NGOs) working in the fields of **climate resilience, disaster preparedness, and community development** often require accurate, timely, and localized rainfall data to inform their interventions, support farmer advisories, and enhance community awareness regarding weather patterns. In many rural and semi-urban areas, such localized data is scarce, leading to challenges in planning community-level climate adaptation and resource management initiatives effectively. Hence, there is a **pressing need for NGOs to have access to systematic rainfall monitoring systems that are community-driven, low-cost, and easily deployable.**

The proposed rainfall mapping system aligns closely with the principles of community work as it **leverages community participation in data collection, integrates open-source GIS-based tools for data visualization, and makes the data accessible for planning and intervention by the NGO and the local community.** This project strengthens the NGO's capacity to engage with the community in climate monitoring, disaster preparedness, and water resource management, thereby enhancing the NGO's role as a facilitator of participatory development.

The current stage of the relationship with the NGO is at the **Initial Design and Testing Phase.** Community members and NGO volunteers are engaged in sharing rainfall data through WhatsApp, which is systematically extracted and visualized through the proposed system. This participatory engagement is being tested to assess data accuracy, community involvement levels, and technical integration with the NGO's operational processes.

The action plan for maintaining the proposed solution at the community level involves **training NGO volunteers to continue collecting rainfall data regularly, maintaining the chat-based data collection system, and using the system to generate regular rainfall maps for community display boards, WhatsApp groups, and local planning meetings.** The NGO will be provided with documentation and step-by-step guides to run the system, ensuring they can update and generate maps independently. Additionally, the system can be scaled by incorporating temperature and humidity data in the future, thereby enhancing its relevance for **local agricultural advisories and disaster preparedness efforts** led by the NGO in collaboration with the community.

EXPECTED DELEVERABLES







Reg56/4- 2005

HUME CENTRE FOR ECOLOGY AND WILDLIFE BIOLOGY

SCIENCE ACROSS BORDERS

PROGRAM AREAS

ECOSYSTEM AND WILDLIFE

MOUNTAIN ECOLOGY
LONG TERM BIRD MONITORING
VULTURE CONSERVATION
WATER BIRDS CONSERVATION
HUMAN ANIMAL INTERACTIONS
ZOOSES RESEARCH

CLIMATE ACTION

WEATHER FORECAST
COMMUNITY WEATHER MONITORING
DISASTER MONITORING
CAPACITY BUILDING OF PRIs
CLIMATE EDUCATION

SCIENCE EDUCATION

SCHOOL SCIENCE PROGRAM
HAMLET LEVEL EDUCATION PROGRAM
KIDS SCIENCE LAB
SCIENCE LIBRARY
WAYANAD ARCHIVES
CAFÉ SCIENTIFIC
THE DISCOURSE

FOOD SYSTEM RESEARCH

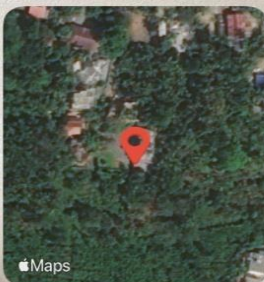
FOOD AND NUTRITION SECURITY
COMMUNITY FOOD SYSTEMS
COMMUNITY SEED SYSTEMS
BIODIVERSITY GOVERNANCE

4th Cross RD. Subhash Nagar, Kalpetta, Kerala, 673121

Email : humecentre@gmail.com Website : www.humecentre.in Contact : 08592959380



GPS Map Camera



Kalpetta, Kerala, India

Pinangode Road, Subhash Nagar, Kalpetta,
Kerala 673121, India

Lat 11.613025, Long 76.075298

06/26/2025 15:21 GMT+05:30

Note : Captured by GPS Map Camera

PROPOSED BUDGET

Sl.No.	Equipment/facilities	Qty.	Total (in Rs)	Justification
1	Software Licenses (Open Source/QGIS, Python libs)	1	0	Use open-source GIS and Python libraries to reduce cost to zero.
2	Internet/Data Charges	1	2,000	Minimum data charges to support uploads and cloud sync.
3	Travel & Local Conveyance	1	3,000	Limited local travel using public transport for field visits.
4	Field Survey Kits (basic stationery, rain gauges)	2	2,000	Basic manual data collection kits.
5	Multimedia Tools (Basic phone camera, free recording apps)	1	1,500	Use existing phone camera and free apps for documentation.
6	Miscellaneous	1	1,000	Minor expenses and consumables.
	Grand Total		9,500	

CONCLUSION AND FUTURE WORK

The GIS-based rainfall mapping system developed for the Balussery Block demonstrates the **practical potential of leveraging open-source technologies and community participation for localized climate data collection and visualization**. By integrating WhatsApp-based community reporting with automated data extraction, grid mapping, and rainfall visualization, the system provides timely and accurate rainfall insights to local stakeholders, NGOs, and community members. This approach not only strengthens climate resilience at the grassroots level but also fosters community awareness and engagement in climate monitoring, aligning with the Sustainable Development Goals related to climate action and community development.

Through the implementation of this system, the NGO and local communities now have **an accessible tool to monitor rainfall trends**, aiding in decision-making for agriculture, water management, and disaster preparedness. The modular design ensures ease of maintenance and future scalability while maintaining low operational costs.

For **future work**, the system can be enhanced by integrating **real-time weather API data to complement community-reported rainfall measurements, enabling cross-validation and improved accuracy**. Expansion to collect **temperature, humidity, and wind speed data** will enable comprehensive micro-climate monitoring in the region. Additionally, developing **mobile-friendly dashboards and automated SMS/WhatsApp notification systems for daily rainfall summaries** will increase accessibility for farmers and community stakeholders. Incorporating **machine learning models to predict short-term rainfall trends** based on historical and real-time data will further strengthen the system's impact on climate adaptation and disaster preparedness. Collaborative efforts with academic institutions and local government bodies can also help in scaling the system to neighboring blocks, thereby **creating a regional climate data network for sustainable planning and community resilience building**.

REFERENCES

1. **ArcGIS Documentation.** (n.d.). *What is GIS?* Retrieved from <https://www.esri.com/en-us/what-is-gis/overview>
2. **Kerala State Disaster Management Authority (KSDMA).** (n.d.). *Rainfall Monitoring and Disaster Management Resources.* Retrieved from <https://sdma.kerala.gov.in>
3. Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment.* Computing in Science & Engineering, 9(3), 90-95. DOI:10.1109/MCSE.2007.55
4. Jordahl, K., et al. (2020). *GeoPandas: Python tools for geographic data.* Retrieved from <https://geopandas.org>
5. Wes McKinney. (2010). *Data Structures for Statistical Computing in Python.* Proceedings of the 9th Python in Science Conference, 51-56. DOI:10.25080/Majora-92bf1922-00a
6. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python.* Journal of Machine Learning Research, 12, 2825-2830.
7. OpenStreetMap Contributors. (n.d.). *OpenStreetMap Data.* Retrieved from <https://www.openstreetmap.org>
8. Google Developers. (n.d.). *Google Translator API Documentation.* Retrieved from <https://cloud.google.com/translate/docs>
9. Kerala State Remote Sensing and Environment Centre (KSREC). (n.d.). *Spatial Data Resources of Kerala.* Retrieved from <http://www.ksrec.kerala.gov.in>
10. United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development.* Retrieved from <https://sdgs.un.org/2030agenda>
11. OGC. (2017). *OpenGIS® Implementation Standard: Simple Feature Access - Part 1: Common Architecture.* Retrieved from <https://www.ogc.org/standards/sfa>
12. Kerala Sasthra Sahithya Parishad (KSSP). (n.d.). *Community Rainfall Monitoring Initiatives.* [Field data collection through WhatsApp groups, Balussery Block].
13. Python Software Foundation. (n.d.). *Python 3.10 Documentation.* Retrieved from <https://docs.python.org/3.10/>